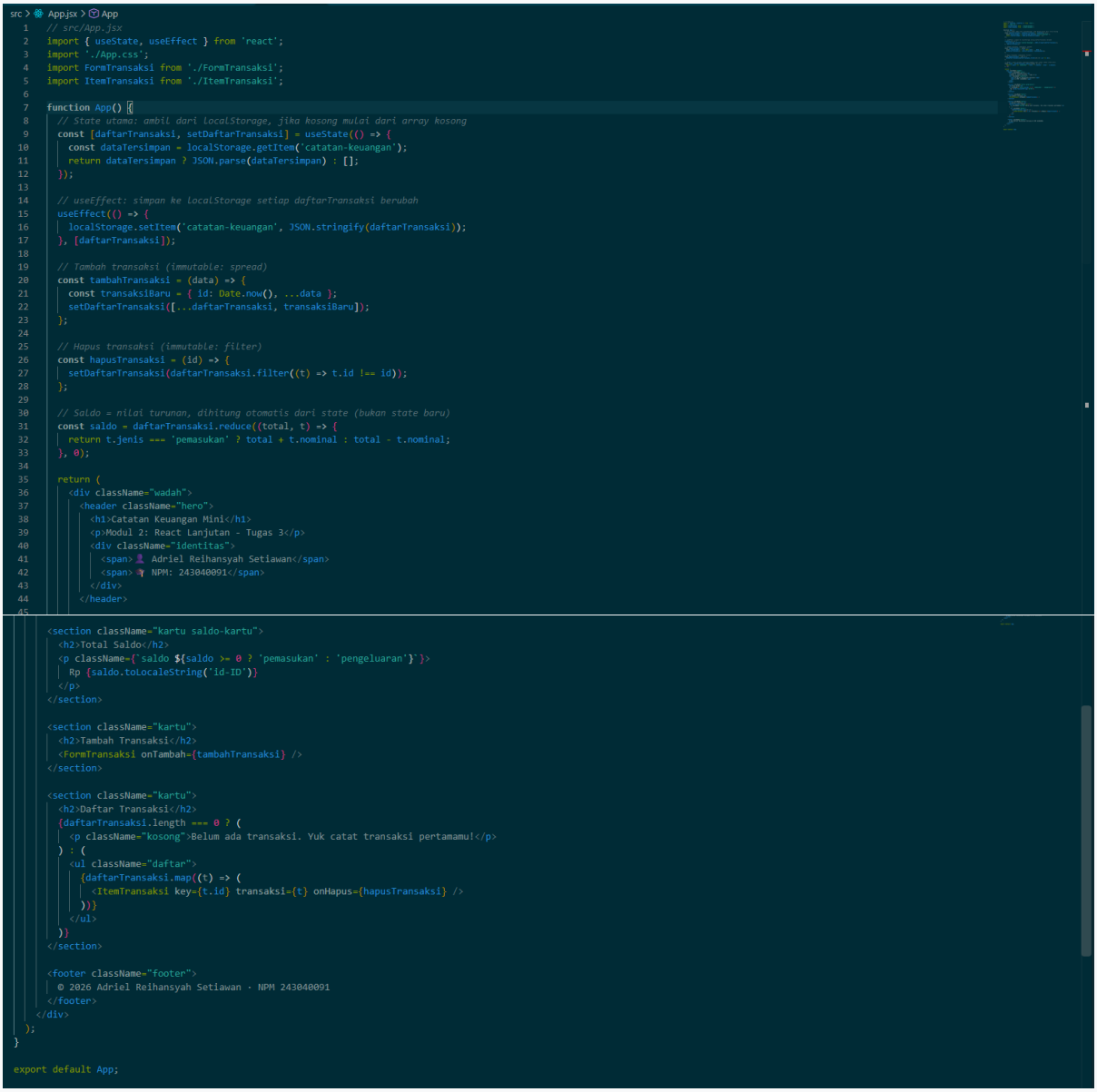
Tugas Pertemuan 3	
IF21W0508 – Praktikum Web	
NPM	243040091
Nama	Adriel Reihansyah Setiawan
Kelas	E
Link Github Repository	https://github.com/AdrielReihansyah/PW2026_243040091

Bagian 1. Screenshot Semua Kode

App.jsx



```jsx
src > App.jsx > App
// src/App.jsx
import { useState, useEffect } from 'react';
import './App.css';
import FormTransaksi from './FormTransaksi';
import ItemTransaksi from './ItemTransaksi';

function App() {
  // State utama: ambil dari localStorage, jika kosong mulai dari array kosong
  const [daftarTransaksi, setDaftarTransaksi] = useState(() => {
    const dataTersimpan = localStorage.getItem('catatan-keuangan');
    return dataTersimpan ? JSON.parse(dataTersimpan) : [];
  });

  // useEffect: simpan ke localStorage setiap daftarTransaksi berubah
  useEffect(() => {
    localStorage.setItem('catatan-keuangan', JSON.stringify(daftarTransaksi));
  }, [daftarTransaksi]);

  // Tambah transaksi (immutable: spread)
  const tambahTransaksi = (data) => {
    const transaksiBaru = { id: Date.now(), ...data };
    setDaftarTransaksi([...daftarTransaksi, transaksiBaru]);
  };

  // Hapus transaksi (immutable: filter)
  const hapusTransaksi = (id) => {
    setDaftarTransaksi(daftarTransaksi.filter((t) => t.id !== id));
  };

  // Saldo = nilai turunan, dihitung otomatis dari state (bukan state baru)
  const saldo = daftarTransaksi.reduce((total, t) => {
    return t.jenis === 'pemasukan' ? total + t.nominal : total - t.nominal;
  }, 0);

  return (
    <div className="wadah">
      <header className="hero">
        <h1>Catatan Keuangan Mini</h1>
        <p>Modul 2: React Lanjutan - Tugas 3</p>
        <div className="identitas">
          <span>👤 Adriel Reihansyah Setiawan</span>
          <span>🪪 NPM: 243040091</span>
        </div>
      </header>

      <section className="kartu saldo-kartu">
        <h2>Total Saldo</h2>
        <p className={`saldo ${saldo >= 0 ? 'pemasukan' : 'pengeluaran'}`}>
          Rp {saldo.toLocaleString('id-ID')}
        </p>
      </section>

      <section className="kartu">
        <h2>Tambah Transaksi</h2>
        <FormTransaksi onTambah={tambahTransaksi} />
      </section>

      <section className="kartu">
        <h2>Daftar Transaksi</h2>
        {daftarTransaksi.length === 0 ? (
          <p className="kosong">Belum ada transaksi. Yuk catat transaksi pertamamu!</p>
        ) : (
          <ul className="daftar">
            {daftarTransaksi.map((t) => (
              <ItemTransaksi key={t.id} transaksi={t} onHapus={hapusTransaksi} />
            ))}
          </ul>
        )}
      </section>

      <footer className="footer">
        © 2026 Adriel Reihansyah Setiawan · NPM 243040091
      </footer>
    </div>
  );
}

export default App;
```

FormTransaksi.jsx

src > FormTransaksi.jsx > default

```jsx
// src/FormTransaksi.jsx
import { useState } from 'react';

function FormTransaksi({ onTambah }) {
  const [keterangan, setKeterangan] = useState('');
  const [nominal, setNominal] = useState('');
  const [jenis, setJenis] = useState('pemasukan');

  // Valid jika keterangan terisi dan nominal lebih dari 0
  const tidakValid = keterangan.trim() === '' || Number(nominal) <= 0;

  const tanganiKirim = (event) => {
    event.preventDefault();
    if (tidakValid) return;
    onTambah({
      keterangan: keterangan.trim(),
      nominal: Number(nominal), // input selalu teks, ubah ke angka
      jenis: jenis,
    });
    setKeterangan('');
    setNominal('');
    setJenis('pemasukan');
  };

  return (
    <form onSubmit={tanganiKirim}>
      <div className="baris" style={{ marginBottom: '8px' }}>
        <input
          className="input"
          type="text"
          placeholder="Keterangan (contoh: Gaji, Makan siang)"
          value={keterangan}
          onChange={(e) => setKeterangan(e.target.value)}
        />
      </div>
      <div className="baris">
        <input
          className="input"
          type="number"
          placeholder="Nominal"
          value={nominal}
          onChange={(e) => setNominal(e.target.value)}
        />
        <select
          className="pilih"
          value={jenis}
          onChange={(e) => setJenis(e.target.value)}
        >
          <option value="pemasukan">Pemasukan</option>
          <option value="pengeluaran">Pengeluaran</option>
        </select>
        <button className="tombol" type="submit" disabled={tidakValid}>
          Tambah
        </button>
      </div>
    </form>
  );
}

export default FormTransaksi;
```

ItemTransaksi.jsx

src > ItemTransaksi.jsx > default

```jsx
// src/ItemTransaksi.jsx
function ItemTransaksi({ transaksi, onHapus }) {
  const adalahPemasukan = transaksi.jenis === 'pemasukan';

  return (
    <li className="item">
      <span>{transaksi.keterangan}</span>
      {/* className dinamis: hijau untuk pemasukan, merah untuk pengeluaran */}
      <span className={`nominal ${adalahPemasukan ? 'pemasukan' : 'pengeluaran'}`}>
        {adalahPemasukan ? '+' : '-'} Rp {transaksi.nominal.toLocaleString('id-ID')}
      </span>
      <button
        className="tombol tombol-merah tombol-kecil"
        onClick={() => onHapus(transaksi.id)}
      >
        Hapus
      </button>
    </li>
  );
}

export default ItemTransaksi;
```

App.css

src > App.css > ...

```css
/* src/App.css */

/* ===== Warna yang dipakai berulang ===== */
:root {
  --biru: #1B365D;
  --biru-terang: #2E5AA8;
  --teks: #1F2937;
  --abu: #6B7280;
  --garis: #E5E7EB;
  --merah: #DC2626;
  --hijau: #16A34A;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  background: linear-gradient(135deg, #EEF2FF 0%, #E0F2FE 100%);
}

.wadah {
  max-width: 640px;
  margin: 0 auto;
  padding: 32px 20px;
  font-family: 'Segoe UI', Arial, sans-serif;
  color: var(--teks);
}

/* ===== Animasi (keyframes) ===== */
@keyframes muncul {
  from {
    opacity: 0;
    transform: translateY(20px);
  }

  to {
    opacity: 1;
    transform: none;
  }
}

@keyframes geser {
  from {
    opacity: 0;
    transform: translateX(-20px);
  }

  to {
    opacity: 1;
    transform: none;
  }
}

@keyframes pop {
  0% {
    transform: scale(0.5);
  }

  70% {
    transform: scale(1.3);
  }

  100% {
    transform: scale(1);
  }
}

@keyframes getar {

  0%,
  100% {
    transform: translateX(0);
  }

  25% {
    transform: translateX(-6px);
  }

  75% {
    transform: translateX(6px);
  }
}
```

App.css > ...

```css
}

/* ===== Header dengan identitas ===== */
.hero {
  background: linear-gradient(135deg, var(--biru), var(--biru-terang));
  color: #FFFFFF;
  padding: 28px 24px;
  border-radius: 16px;
  margin-bottom: 24px;
  box-shadow: 0 10px 25px -5px rgba(27, 54, 93, 0.4);
  animation: muncul 0.6s ease;
}

.hero h1 {
  margin: 0 0 4px;
  font-size: 26px;
}

.hero p {
  margin: 0 0 16px;
  opacity: 0.85;
}

.identitas {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}

.identitas span {
  background: rgba(255, 255, 255, 0.18);
  padding: 6px 12px;
  border-radius: 999px;
  font-size: 13px;
}

/* ===== Kartu ===== */
.kartu {
  background: #FFFFFF;
  border-radius: 14px;
  padding: 20px;
  margin-bottom: 20px;
  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.08);
  animation: muncul 0.5s ease backwards;
  transition: transform 0.2s, box-shadow 0.2s;
}
```

src > App.css > ...

```
  }

  .kartu:nth-of-type(2) {
    animation-delay: 0.1s;
  }

  .kartu:nth-of-type(3) {
    animation-delay: 0.2s;
  }

  .kartu:hover {
    transform: translateY(-3px);
    box-shadow: 0 12px 20px rgba(0, 0, 0, 0.12);
  }

  .kartu h2 {
    margin-top: 0;
    font-size: 18px;
    color: var(--biru);
  }

  /* ===== Form & tombol ===== */
  .baris {
    display: flex;
    gap: 8px;
  }

  .input,
  .pilih {
    flex: 1;
    padding: 10px 12px;
    border: 1px solid #D0D7DE;
    border-radius: 8px;
    font-size: 14px;
    transition: border-color 0.2s, box-shadow 0.2s;
  }

  .pilih {
    flex: none;
  }

  .input:focus,
  .pilih:focus {
    outline: none;
    border-color: var(--biru-terang);
    box-shadow: 0 0 0 3px rgba(46, 90, 168, 0.2);
  }
```

```
.item {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 12px 8px;
  border-bottom: 1px solid var(--garis);
  border-radius: 8px;
  animation: geser 0.3s ease;
  transition: background 0.2s;
}

.item:hover {
  background: #F8FAFC;
}

.item span {
  flex: 1;
  transition: color 0.3s;
}

.item span.nominal {
  flex: none;
}

.item input[type="checkbox"] {
  width: 18px;
  height: 18px;
  accent-color: var(--biru);
  cursor: pointer;
}

.selesai {
  text-decoration: line-through;
  color: #9CA3AF;
}

.kosong {
  color: var(--abu);
  text-align: center;
  font-style: italic;
  animation: muncul 0.4s ease;
}
```

```css
.galat {
  color: var(--merah);
  font-size: 13px;
  margin: 6px 0 0;
  animation: getar 0.4s;
}

.ringkasan {
  font-size: 14px;
  color: #374151;
  margin: 0;
}

.filter-baris {
  margin-bottom: 12px;
}

.aksi {
  margin-top: 12px;
  animation: muncul 0.3s ease;
}

/* ===== Penghitung & progres ===== */
.angka {
  display: inline-block;
  font-size: 20px;
  font-weight: bold;
  color: var(--biru-terang);
  animation: pop 0.3s ease;
}

.progres {
  height: 8px;
  background: var(--garis);
  border-radius: 999px;
  overflow: hidden;
  margin: 8px 0 16px;
}

.isi-progres {
  height: 100%;
  background: linear-gradient(90deg, #22C55E, #16A34A);
  transition: width 0.5s ease;
}
```

```css
/* ===== Khusus Catatan Keuangan (Tugas 3) ===== */
.saldo-kartu {
  text-align: center;
}

.saldo {
  font-size: 32px;
  font-weight: bold;
  margin: 0;
}

.pemasukan {
  color: var(--hijau);
  font-weight: bold;
}

.pengeluaran {
  color: var(--merah);
  font-weight: bold;
}

/* ===== Footer ===== */
.footer {
  text-align: center;
  color: var(--abu);
  font-size: 13px;
  margin-top: 24px;
}
```

Bagian 2. Penjelasan Kode

1. App.jsx

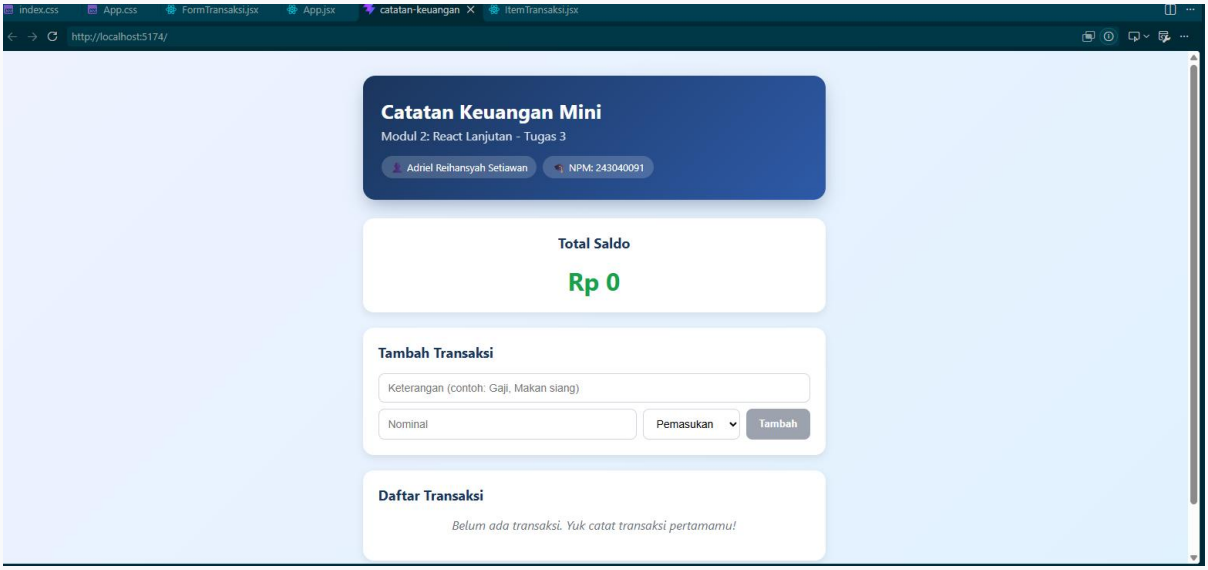
Ini komponen induk yang menyimpan daftar transaksi memakai useState. Data awalnya diambil dari localStorage, dan setiap daftar berubah, useEffect menyimpannya kembali ke localStorage supaya tidak hilang saat refresh. Fungsi tambahTransaksi memakai spread (...) dan hapusTransaksi memakai .filter(), jadi keduanya membuat array baru. Total saldo dihitung dengan .reduce(): pemasukan ditambah, pengeluaran dikurangi. Saldo tidak dibuat sebagai state karena bisa dihitung dari daftar transaksi. Jika belum ada transaksi, ditampilkan pesan khusus.

2 FormTransaksi.jsx
Formulir untuk menambah transaksi, berisi keterangan, nominal, dan jenis (Pemasukan atau Pengeluaran). Ketiganya disimpan di state dan dihubungkan dengan value dan onChange (controlled form). Tombol Tambah nonaktif jika keterangan kosong atau nominal belum lebih dari 0. Nominal diubah ke angka dengan Number(nominal), karena isi input selalu berupa teks. Tanpa itu, “5000” + “2000” akan menjadi “50002000”. Setelah dikirim, komponen ini memanggil fungsi onTambah dari App.

3. ItemTransaksi.jsx
Komponen untuk menampilkan satu transaksi beserta tombol Hapus. Pemasukan ditampilkan hijau dengan tanda plus, sedangkan pengeluaran merah dengan tanda minus, memakai className dinamis. Tombol Hapus memanggil fungsi onHapus dari App.

4. App.css
Berisi gaya tampilan dan animasi sederhana, serta warna khusus untuk saldo, pemasukan, dan pengeluaran.
Kesesuaian dengan ketentuan tugas
(a) Formulir terkendali: FormTransaksi.jsx.
(b) Daftar transaksi dan tombol hapus: App.jsx dan ItemTransaksi.jsx.
(c) Saldo otomatis: variabel saldo dengan .reduce().
(d) Warna hijau dan merah: className dinamis di ItemTransaksi.jsx.
(e) Pesan jika kosong: ternary di App.jsx.
(f) Simpan ke localStorage: useEffect di App.jsx.
Jumlah komponen: 3 (App, FormTransaksi, ItemTransaksi).

Bagian 3. Screenshot Hasil Antarmuka



Bagian 4. Penjelasan Hasil Antarmuka

1. Tampilan awal: Saldo Rp 0 dan muncul pesan “Belum ada transaksi” karena daftar masih kosong.
2. Tombol nonaktif: Tombol Tambah berwarna abu-abu selama keterangan atau nominal belum diisi dengan benar.
3. Tambah pemasukan: Transaksi muncul dengan nominal hijau dan tanda plus, lalu saldo bertambah.
4. Tambah pengeluaran: Transaksi muncul dengan nominal merah dan tanda minus, lalu saldo berkurang.
5. Beberapa transaksi: Saldo adalah total pemasukan dikurangi total pengeluaran, dan dihitung otomatis dari daftar.
6. Saldo negatif: Jika pengeluaran lebih besar dari pemasukan, saldo berubah menjadi merah.
7. Hapus transaksi: Transaksi hilang dari daftar dan saldo langsung dihitung ulang.
8. Setelah refresh: Data tetap ada karena sudah disimpan di localStorage lewat useEffect.
9. Local Storage: Di tab Application terlihat kunci catatan-keuangan yang berisi data transaksi dalam format JSON.